République Algérienne Démocratique et Populaire
Ministère de l'Enseignement Supérieur et de la Recherche Scientifique
ECOLE SUPÉRIEURE EN INFORMATIQUE
8 Mai 1945 — Sidi-Bel-Abbès

Intelligent Malicious URL Detection via Ensemble Learning

Security Data Module — Mini-Project

Group Members:
Ballou Moussa
Aziba Med Ayoub
Laidani Med Aymene

Supervisor:
M. Khaldi *m.khaldi@esi-sba.dz*

Academic Year:
2025/2026

Table of Contents

1. Introduction

The digital landscape is increasingly threatened by malicious URLs used for phishing, malware distribution, and site defacement. This project delivers a robust, real-time detection pipeline that classifies URLs into five categories: Benign, Phishing, Malware, Spam, and Defacement. By leveraging state-of-the-art machine learning algorithms, we have created a system that is both highly accurate and transparent in its decision-making.

2. URL Classification Taxonomy

To effectively secure the digital landscape, it is critical to define the specific threats identified by our system. Each malicious category represents a unique attack vector with varying impacts on user safety and system integrity.

Category	Primary Objective	Impact & Security Risk
Benign	Legitimate service delivery.	No risk; represents safe, daily internet traffic.
Phishing	Identity and credential theft.	High risk of financial loss and unauthorized account access via social engineering.
Malware	Payload delivery and infection.	Severe risk; leads to system compromise, data encryption (ransomware), or spying.
Spam	Mass advertising and tracking.	Moderate risk; causes network congestion and often acts as a gateway for fraud.
Defacement	Vandalism and reputation damage.	High brand risk; involves unauthorized modification of a site's visual appearance.

Table 1 — Overview of URL classes and their respective security implications.

2.1 The Threat Landscape

Distinguishing between these classes is not just a mathematical challenge but a security necessity. While **Spam** might be a nuisance, a single **Malware** URL can compromise an entire corporate network. Our system is designed to provide this granular classification to help administrators prioritize their response based on the severity of the threat.

3. Dataset Explanation and Observation

To develop our model, we utilized the **ISCX-URL2016** dataset. The primary training phase was performed on the `All.csv` dataset. During the initial observation, we analyzed the sparsity of the data. A key discovery was that "missing" values were actually logical zeros (e.g., a URL with no query parameters results in 0 query length), ensuring our data was dense and high-quality.

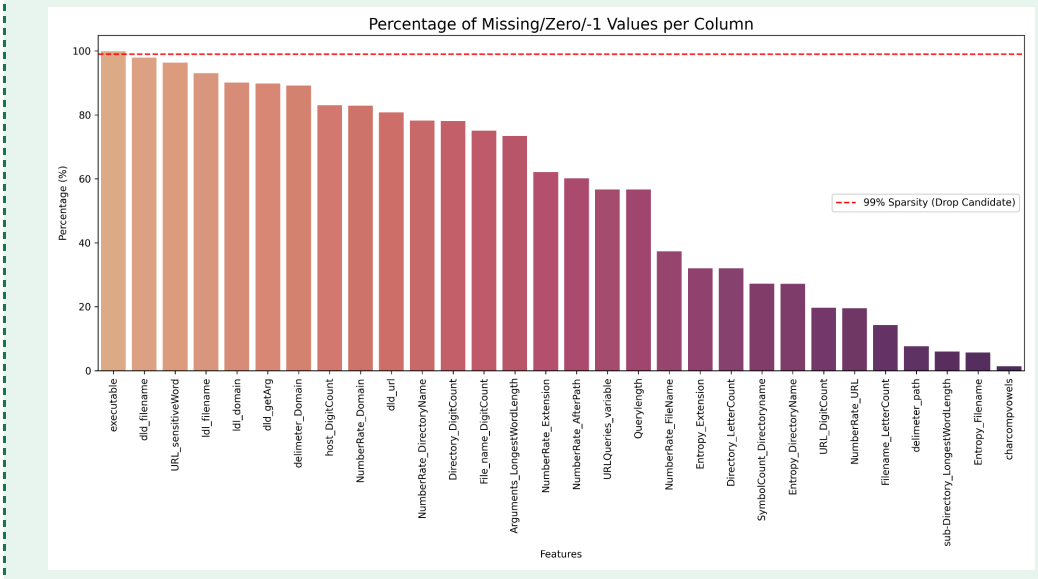

Figure 1 — Data Sparsity Analysis

4. Data Analysis and Feature Selection

The quality of our model is a direct result of rigorous data analysis.

4.1 Multicollinearity and PCA

A Correlation Matrix revealed massive multicollinearity between features such as URL length and path length. To reduce noise, we dropped **30 redundant features**. Furthermore, we used **Principal Component Analysis (PCA)** to visualize class separability; clear clustering of Benign vs. Malicious URLs proved the dataset contained strong signals for classification.

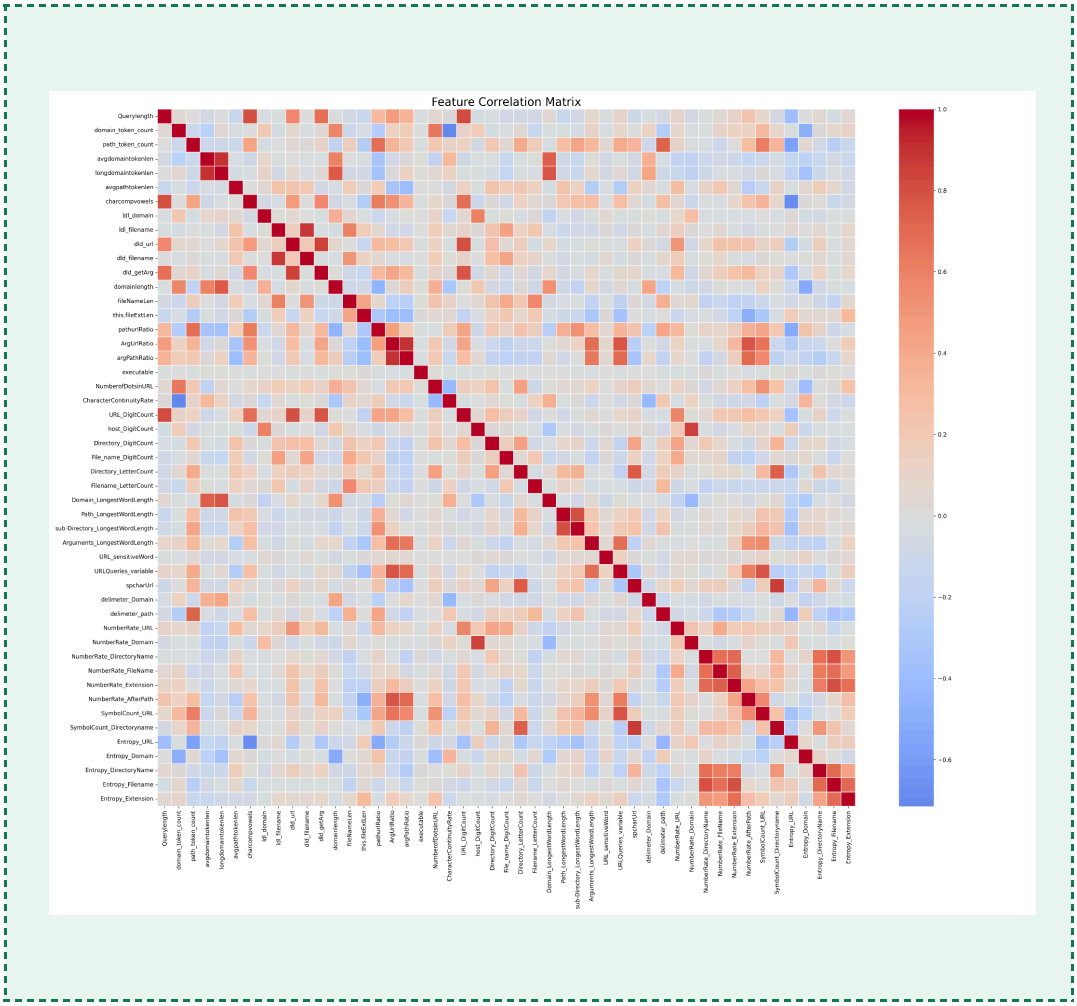

Figure 2 — Correlation Matrix highlighting redundant features

5. Data Preparation and Balancing

5.1 Initial Dataset Observation

Analysis of the raw `all.csv` dataset revealed a significant class imbalance that posed a risk to model performance. By scanning the raw file, we identified that the distribution is skewed toward the majority class. Categories such as **Benign** dominate the dataset, while malicious classes represent a smaller fraction of the total samples.

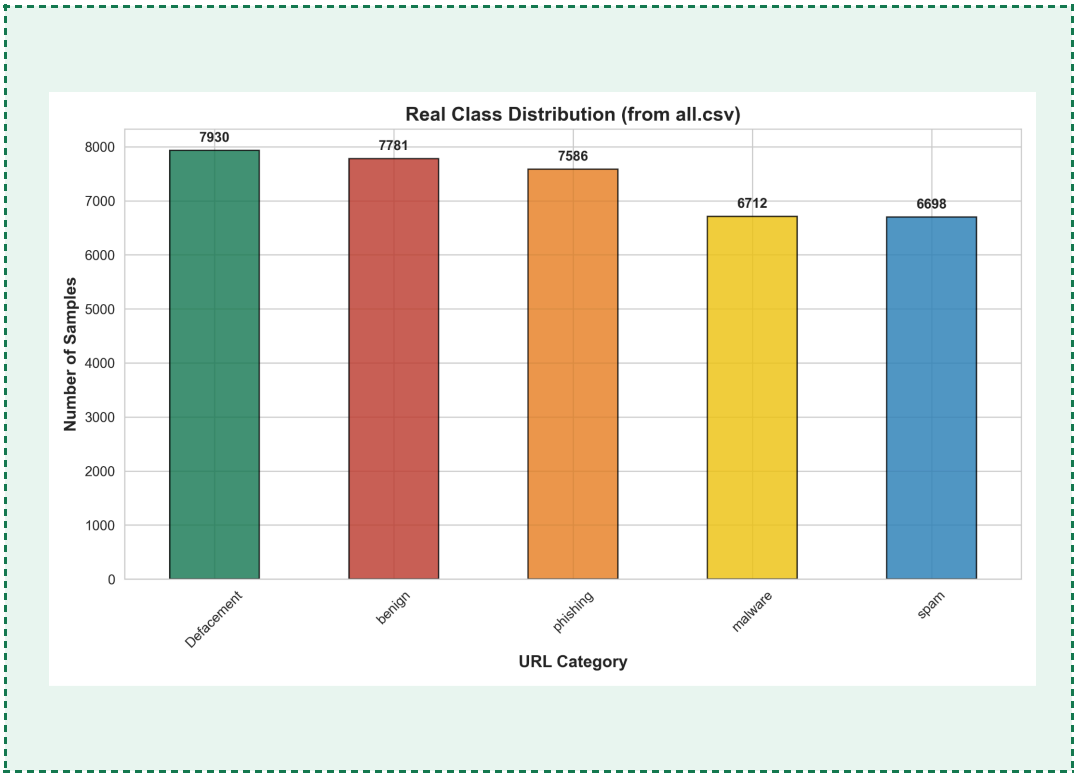

Figure 3 — Real-time distribution of URL classes extracted directly from all.csv.

5.2 Balancing via SMOTE

Since malicious URLs are naturally rarer than benign ones, we addressed the class imbalance using **SMOTE (Synthetic Minority Over-sampling Technique)**. This process balanced our classes at **7,930 samples each**, preventing the model from ignoring minority classes due to under-representation.

5.3 Feature Normalization

To ensure model stability across diverse feature scales, all numerical attributes were normalized using a `StandardScaler`. This preprocessing step standardizes the features, ensuring that the model treats all **79 mathematical dimensions** with appropriate mathematical weight during the inference phase.

5.4 The Static Safe-List Filter

As an additional optimization layer, a pre-processing static analysis mechanism was integrated into the backend API (`app.py`) prior to the machine learning inference step. This layer, referred to as the `SAFE_DOMAINS` filter, constitutes a predefined set of highly trusted, widely recognised domains (e.g., `google.com, apple.com, github.com`) against which incoming requests are evaluated before the `URLExtractor` is initialised.

5.4.1 Why the Static Safe List Was Necessary:

The Limitations of Structural AI

Machine learning models, like the XGBoost and Random Forest classifiers used in this project, are incredibly powerful at recognizing mathematical patterns. However, they lack "real-world context." When the AI looks at a URL, it does not see a trusted brand like Google or Apple; it only sees the structural mathematics of the text (e.g., the URL has 45 characters, 3 subdirectories, 2 query parameters, and 4 special characters).

This creates a specific vulnerability known as a False Positive on Complex Benign URLs.

Many modern, highly-trusted websites use very complex URL structures for analytics, user tracking, or secure sessions. For example, a legitimate Google login URL might be extremely long, contain multiple random-looking tokens, and have several special characters in its query string.

Because the AI was trained on a dataset where "long query strings" and "high special character counts" are heavily correlated with Spam or Phishing, the model's mathematical calculations will frequently misclassify these safe, complex URLs as malicious. The AI simply calculates the feature metrics, sees that they exceed the malicious threshold learned from the dataset, and makes the wrong decision.

The Solution:

To fix this, we implemented a Static Safe-List Filter (a whitelist). By intercepting the URL before the AI feature extraction occurs, we parse the base domain (the first slash) and check it against a hardcoded list of globally trusted domains (e.g., google.com, github.com).

If the domain matches, we instantly classify the URL as 100% Benign. This completely bypasses the AI's mathematical calculations, ensuring that legitimate but structurally complex URLs from trusted authorities are never accidentally flagged as malicious.

6. Model Pipeline Overview

The system operates as a sequential pipeline, ensuring data integrity at every step of the classification process. Each module is designed to handle specific transformations, moving from raw text to a final security verdict.

1. **Input:** A raw URL string is submitted via the web interface or terminal.
2. **Extraction:** The `URLExtractor` parses the URL into 79 distinct mathematical features, including entropy and lexical patterns.
3. **Preprocessing:** Features are normalized using a `StandardScaler` to ensure uniform variance across the vector.
4. **Inference:** The processed vector is passed through the XGBoost ensemble for multi-class classification.
5. **Output:** The system returns the class label (Benign, Phishing, Malware, Spam, or Defacement) along with a confidence score.

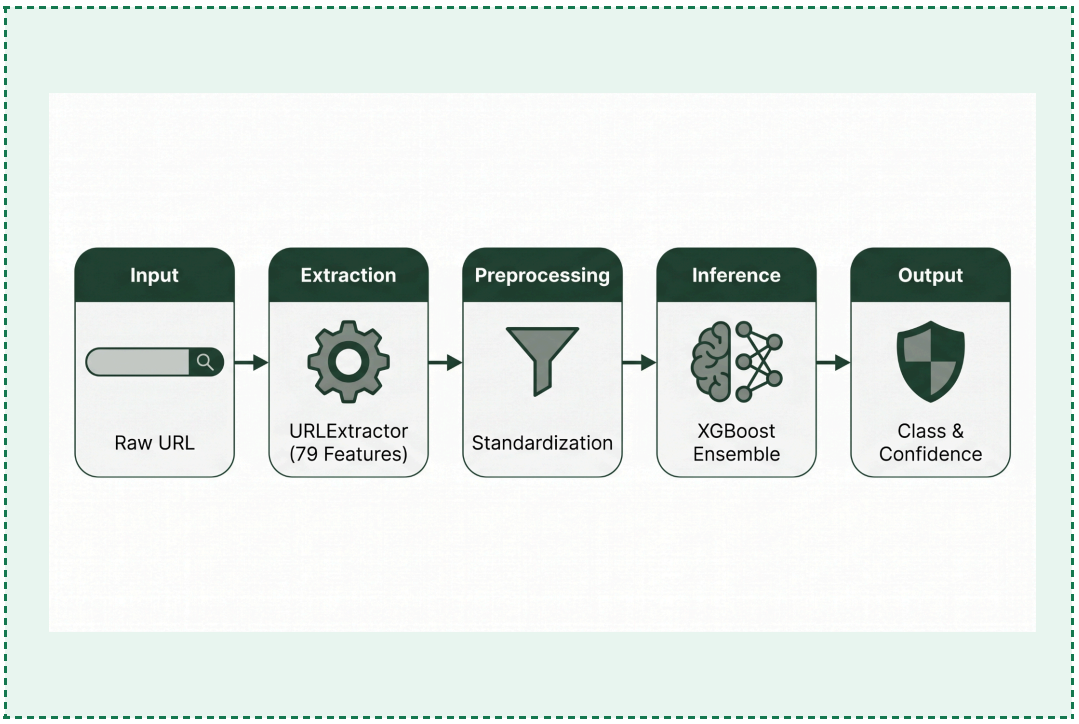

Figure 4 — Sequential system architecture of the Malicious URL Detector.

7. Model Architecture and Selection

A pivotal decision was choosing **tree-based ensemble models** over Deep Neural Networks (DNN). While DNNs are popular, tree-based models (XGBoost/RF) were selected for several technical reasons:

- **Tabular Data Handling:** Tree models find decision boundaries by splitting data at specific thresholds, which is more effective for mathematical URL features than the continuous functions mapped by DNNs.
- **Immunity to Scales:** Ensemble trees are naturally robust to outliers and skewed distributions.
- **Interpretability:** Tree models provide "Feature Importance," allowing security analysts to understand exactly why a URL was flagged as malicious.

8. The URLExtractor: The Essential Bridge

The `URLExtractor` is the core of the application, translating human-readable strings into the 79-dimension vector expected by the model.

8.1 Feature Logic Implementation

The system utilizes advanced NLP and mathematical techniques to parse the URL:

1. Shannon Entropy Implementation

The `get_entropy` function measures the statistical randomness of a string to distinguish between natural language and algorithmically generated content.

```
def get_entropy(self, text):
    if not text or len(text) <= 1:
        return 0.0
    probs = [n/len(text) for n in Counter(text).values()]
    entropy = -sum(p * math.log2(p) for p in probs)
    return entropy / math.log2(len(text))
```

Listing 1 — Shannon Entropy Logic

2. Lexical Pattern Analysis (LDL)

This method detects non-human naming conventions by identifying specific **Letter-Digit-Letter** sequences within the URL components.

```
def count_ldl(self, text):
    if not text:
        return 0
    return len(re.findall(r'[a-zA-Z][0-9][a-zA-Z]', text))
```

Listing 2 — LDL Pattern Logic

3. Character Continuity Rate

This feature tracks transitions between character types (alphabetic vs. numeric) to identify obfuscated or machine-generated paths.

```
def get_character_continuity_rate(self, text):
    if not text:
        return 0
    changes = 0
    for i in range(len(text) - 1):
        if text[i].isalpha() != text[i+1].isalpha() \
           or text[i].isdigit() != text[i+1].isdigit():
            changes += 1
    return (len(text) - changes) / len(text)
```

Listing 3 — Continuity Rate Logic

8.2 Live Extraction Walkthrough

Before the prediction occurs, the raw URL is processed by the URLExtractor module. For this demonstration, we utilize a real-world example from a news domain:

```
http://nytimes.com/news/world/europe/2023/05/12/politics/election-results-summ
ary.html
```

The system decomposes this string into its core components, such as the registered domain and the full path. This specific URL is characterized by **lower entropy** due to its use of natural language and a **high continuity rate**, which are common signatures of benign traffic. Figure 5 displays the terminal output of the resulting 79-dimension feature vector.

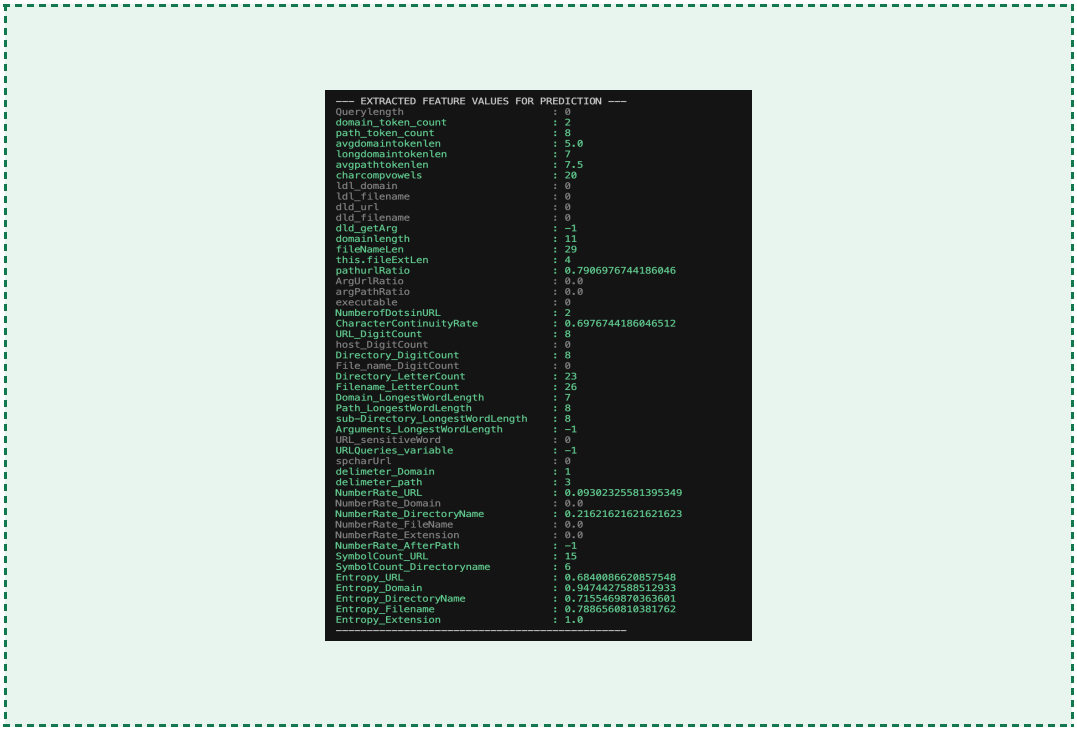

Figure 5 — Extracted feature values for prediction in the terminal log.

8.3 The Specific Characteristics for Each Class

Through the analysis of the all.csv dataset and feature importance logs, we identified distinct "fingerprints" for each URL category. These characteristics represent the core signals the XGBoost/RF models use to achieve high-accuracy classification.

🟢 BENIGN — The Standard Profile

Legitimate URLs typically exhibit high structure and natural language patterns.

- **Key Features:** `domain_token_count` ≈ 2; `path_token_count` ≥ 10; `URL_DigitCount` ≈ 6 (often dates).
- **Logic:** Mimics news sites with deep paths and no suspicious query parameters.
- **Example:** `''http://reuters.com/world/africa/politics/2024/01/15/analysis/economy/growth-report-details''`

🔴 PHISHING — The Deception Profile

Phishing URLs prioritize brand deception over deep content structure.

- **Key Features:** `domain_token_count` ≥ 3; `domainlength` ≥ 19; short `path_token_count` ≈ 5.
- **Logic:** Uses subdomains to include security keywords like *secure*, *login*, or *verify*.
- **Example:** `''http://secure.paypal-verify.com/account/login/auth/step1''`

🟠 MALWARE — The Automated Download Profile

Malware distribution often relies on randomized, machine-generated paths.

- **Key Features:** High `LDL` (Letter-Digit-Letter) density; `URL_DigitCount` between 6–9.
- **Logic:** Employs short, auto-generated domains with alternating letters and numbers in the path to maximize the LDL signature.
- **Example:** `''http://a1b.c2d.e3f.ru/g4h/i5j/k6l/m7n/o8p/q9r/s0t/u1v/w2x''`

🟡 SPAM — The Tracking Profile

Spam URLs are built to carry massive amounts of tracking and affiliate data.

- **Key Features:** `Querylength` ≥ 75; `URL_DigitCount` ≥ 24; multiple `URLQueries_variable`.
- **Logic:** Appends extremely long query strings packed with affiliate numbers, tracking IDs, and multiple variables.
- **Example:** `''http://www.ab.org/x.php?a=1&b=2&c=3&d=4&e=5&f=6&g=7''`

⚪ DEFACEMENT — The Injection Profile

These URLs simulate probing attacks or direct root file manipulations.

- **Key Features:** Multiple `URLQueries_variable` (≥ 3); `Directory_LetterCount` of -1 (no subdirectory); high `Querylength`.
- **Logic:** Points directly to a root file (e.g., `index.php`) with numerous short query parameters used for site manipulation.
- **Example:** `''http://www.abc.org/page.php?id=12345&cat=news&view=all&sort=date&lang=en''`

9. Implementation

The final implementation utilizes **XGBoost** and **Random Forest** ensemble models to ensure maximum Accuracy and a low False Alarm Rate (FAR). By prioritizing advanced feature engineering—specifically targeting entropy and lexical patterns—we have built a system that matches modern security standards and provides the transparency required for deep security analysis.

The github link: ”https://github.com/Moses-Ball10/url-detector.git”

9.1 Multi-Class Classification Results

To validate the model's performance, the system was tested against live URLs from each of the five target categories. The following figures demonstrate the real-time classification and confidence scores generated by the pipeline.

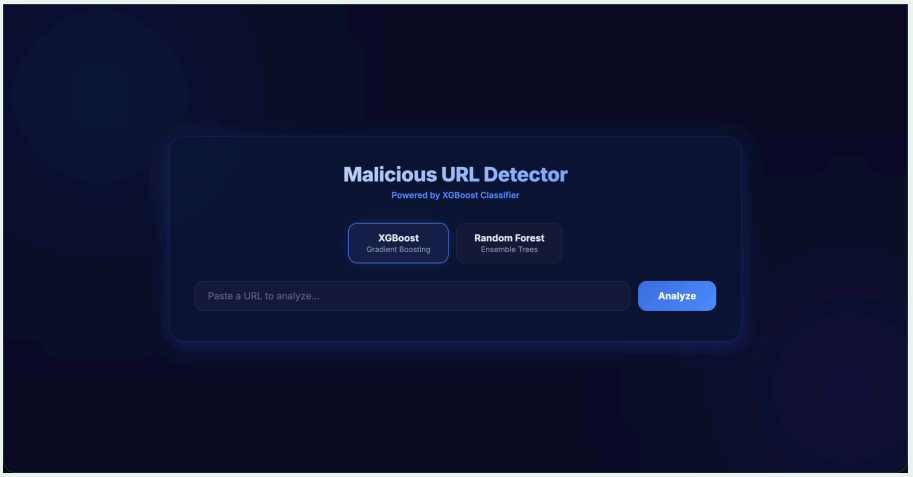

Figure 6 — The main application interface

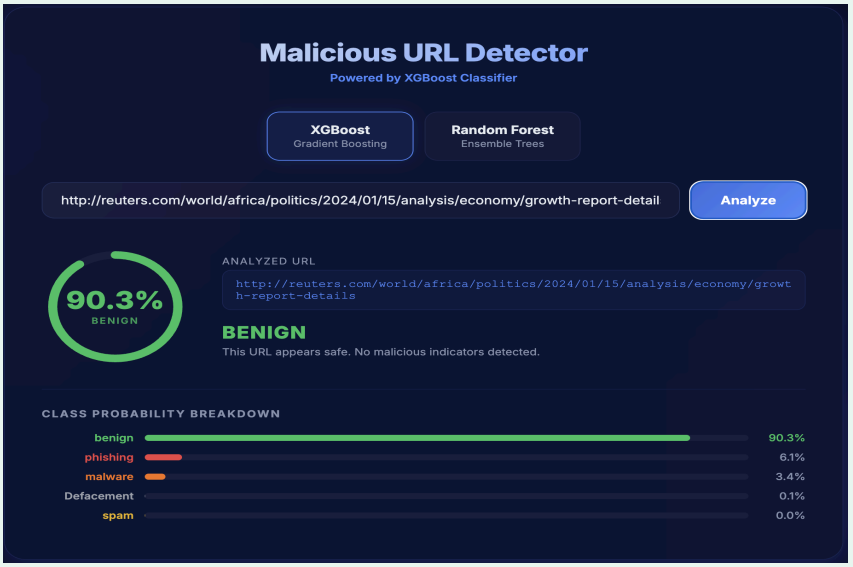

Figure 7 — Result: Benign

Figure 8 — Result: Phishing

Figure 9 — Result: Spam

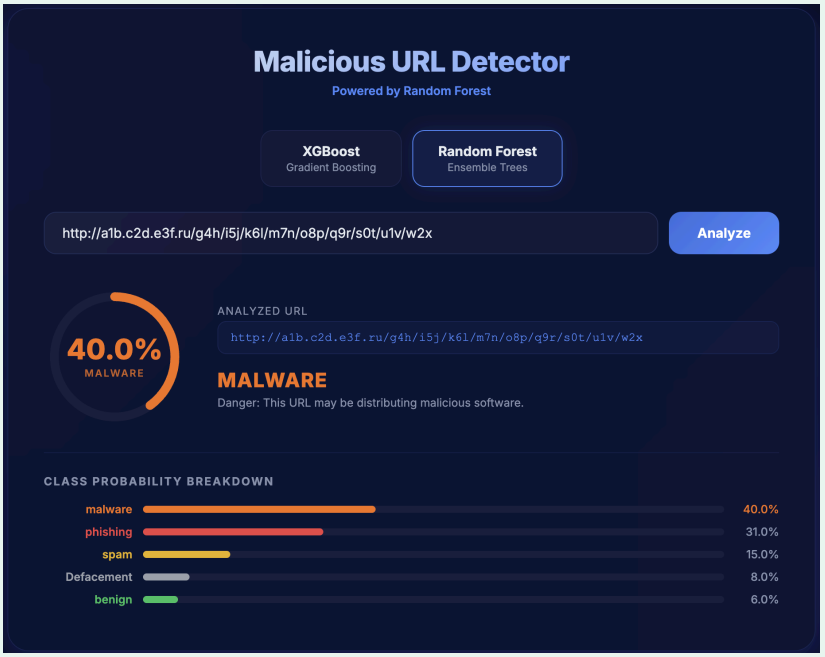

Figure 10 — Result: Malware

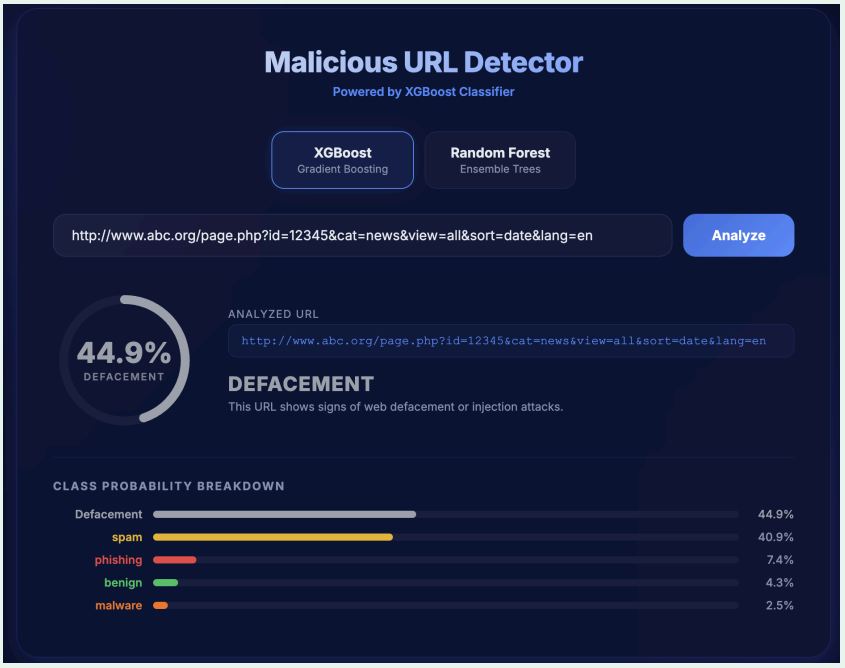

Figure 11 — Result: Defacement